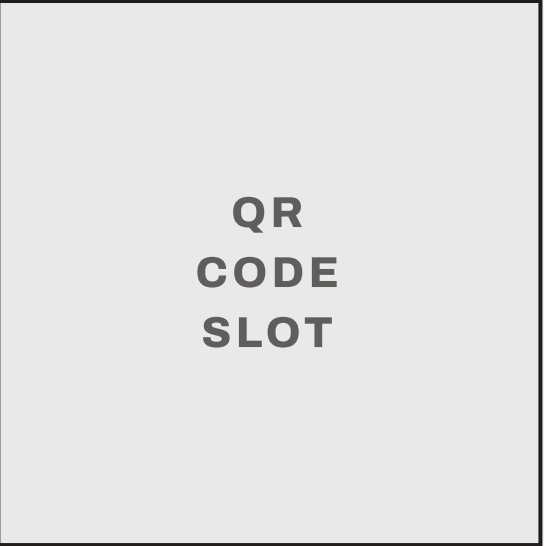


Benchmarking Rust and C Math Libraries for Embedded Robotics

Rust and C math libraries for embedded robot firmware, benchmarked on real hardware across five chips.

Pascal Harprecht* · Thomas Lübbehüsen* · Wolfgang Hönig — Technical University of Berlin · *equal contribution



PAPER & CODE

01 AN OPEN HARDWARE-IN-THE-LOOP BENCHMARK

One task definition, every platform and library

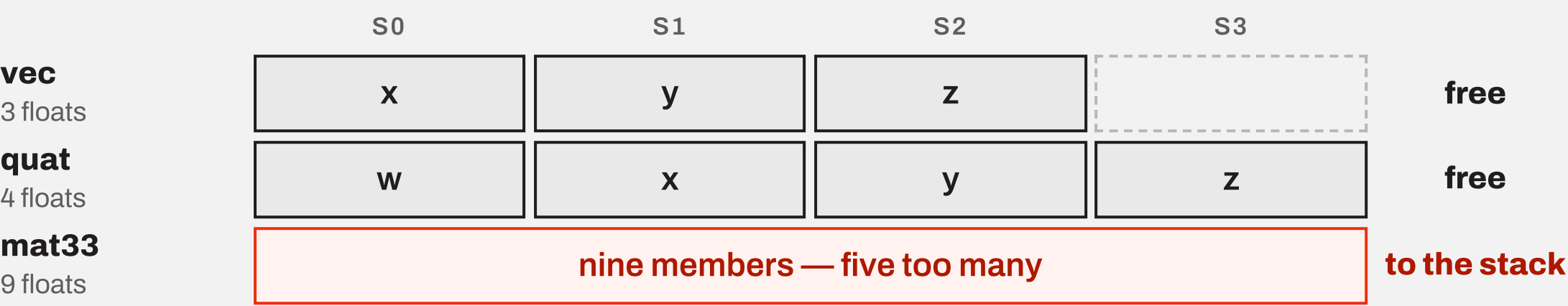
18 × 909 math primitives × input instances, baked into flash by a proc-macro at compile time	6 libraries glam, nalgebra, micromath, libm vs. CMSIS-DSP and cmath3d (Crazyflie)	5 chips STM32F405, RP2040, RP2350, ESP32-S3, nRF52840 — real boards, not simulated	Cycle-exact on-chip counters (DWT, SysTick, CCOUNT) streamed over RTT — no RTOS jitter
--	---	--	--

02 WHAT CROSSES THE BOUNDARY

STM32F405 · Cortex-M4F, 168 MHz

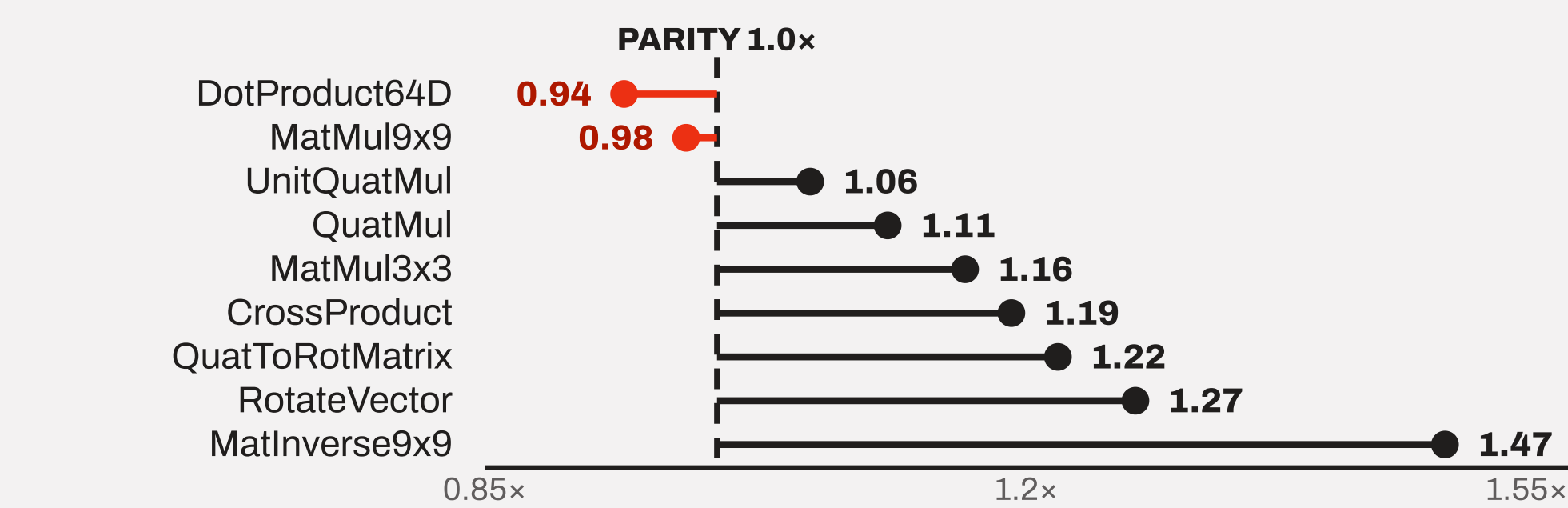
The **shape** of the data crossing the boundary decides the cost. A 3×3-matrix result is mostly an ABI number, not a language one.

WHY: THERE ARE ONLY FOUR SLOTS



ARM's calling convention hands a struct over in floating-point registers on one condition: every member is a float, and there are **at most four of them**.

Copied to the stack going in, returned through a hidden pointer coming out — traffic Rust never pays. The ratio tracks that copying almost exactly: **1.06–1.18×** with none of it, **2.76×** when half the wrapper is marshalling. A mat33 row is an ABI result, not a language one.



ACCOUNT FOR THE SHAPE AND ARITHMETIC IS A TIE

All nine register- and pointer-passed primitives, each read at the profile where its C-vs-Rust number is actually valid. The whole band is **0.94×** to **1.47×**, and glam and nalgebra sit within percent-level of each other.

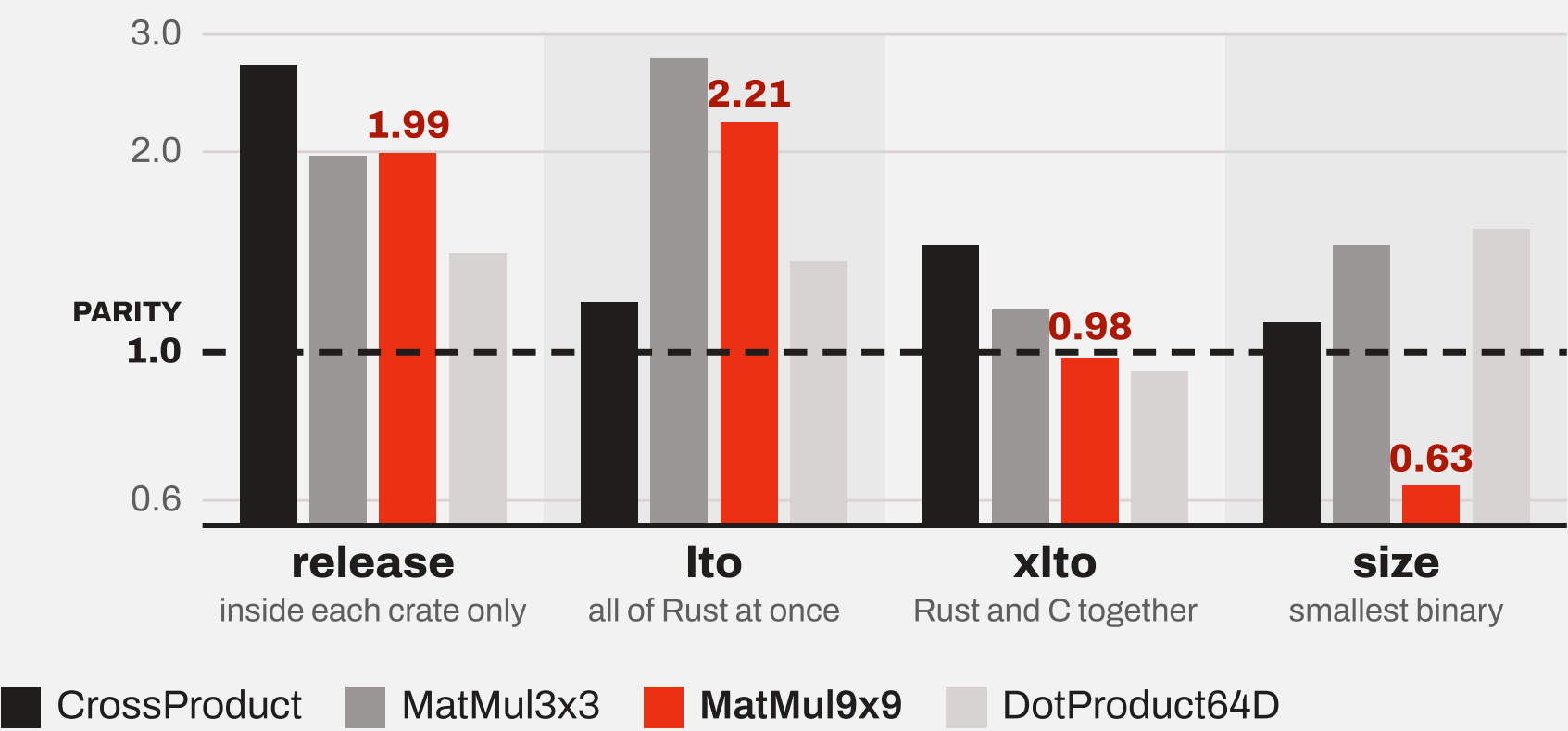
■ Rust ahead ■ C ahead

03 HOW FAR THE OPTIMIZER CAN SEE

one representative task per class of operands

There is no single right build profile: lto is the fair reading for one class of operands, xlto for the other two.

C time ÷ fastest Rust time, per build profile



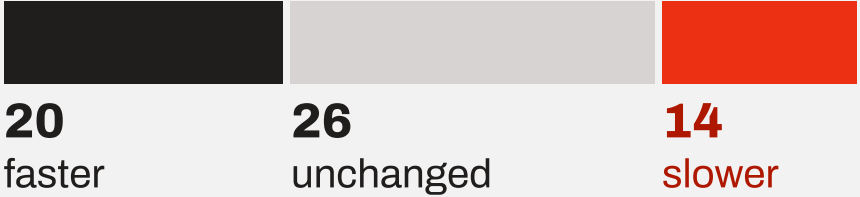
9×9 matrix multiplication travels from **2.21×** in Rust's favour to **0.63×** in C's on nothing but the link strategy. **lto** and **xlto** are this project's own names.

WHICH PROFILE MAKES THE COMPARISON FAIR DEPENDS ON THE OPERANDS

HOW THE OPERANDS ARE PASSED	release	lto	xlto	size	WHY THE OTHERS MISLEAD
all in registers CrossProduct, QuatMul, UnitQuatMul, RotateVector	2.70	1.19 READ HERE	1.45	1.11	release: Rust's wrapper stays un-inlined. xlto: C's own call site slows 26%.
a 3×3 matrix by value MatMul3x3, QuatToRotMatrix	1.97	2.76	1.16 READ HERE	1.45	lto bills C for stack copies Rust never makes.
a pointer to a big buffer MatMul9x9, MatInverse9x9, DotProduct64D	1.99	2.21	0.98 READ HERE	0.63	lto optimizes Rust fully, the C object not at all. size discards nalgebra's unrolling.

AND XLTO IS NOT A STRICT WIN

all 60 task × library pairs that ran under both, on the STM32F405



Pure-Rust rows regress too — micromath SinCos +31%, nalgebra MatMul9x9 +20%. Not only an FFI story.

Reach for xlto deliberately. A clear win for 3×3-by-value code and CMSIS-DSP's larger routines; close to a coin flip elsewhere.

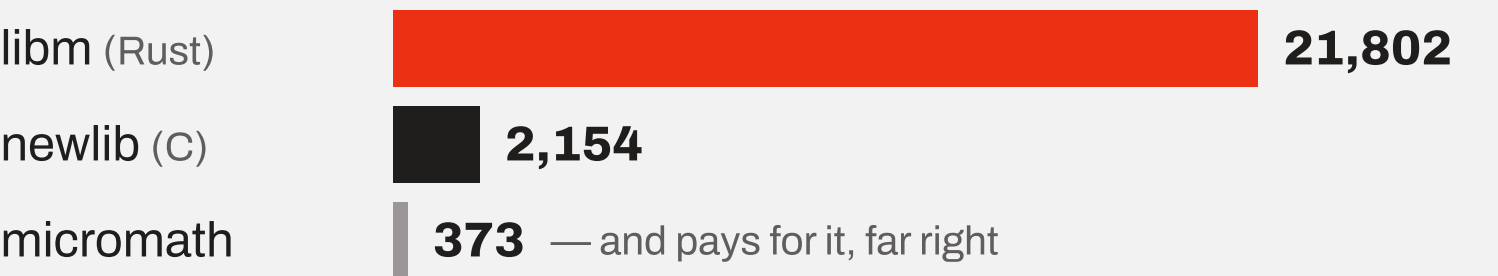
04 WHICH BACKEND YOU LINK

lto profile · median ns

Same operation, three implementations, a 10× spread — and the gap tracks the library, not the language.

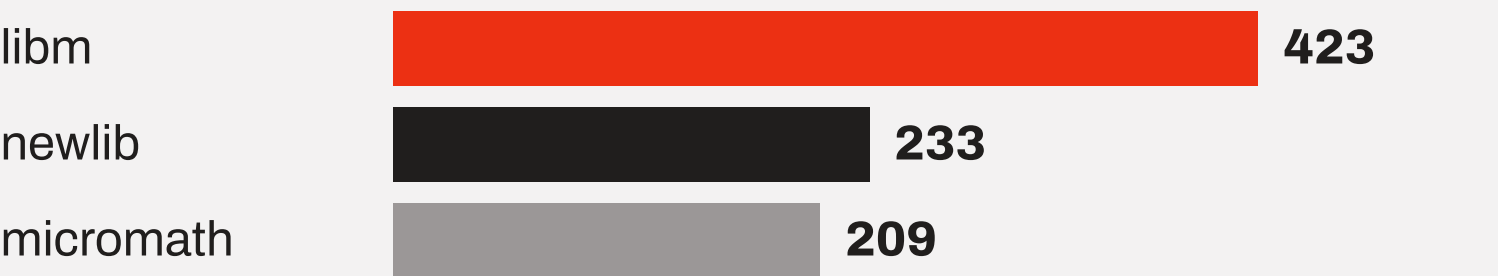
SinCos one sine/cosine pair

10.1× spread



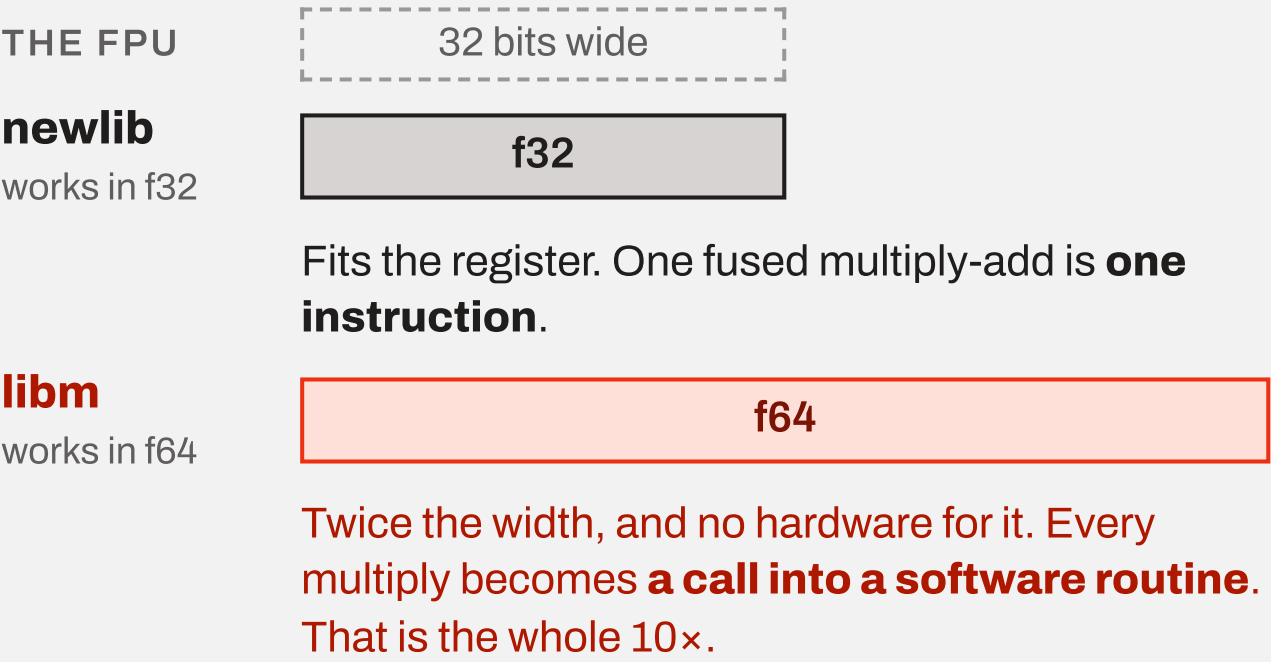
Sqrt one square root

1.82× spread



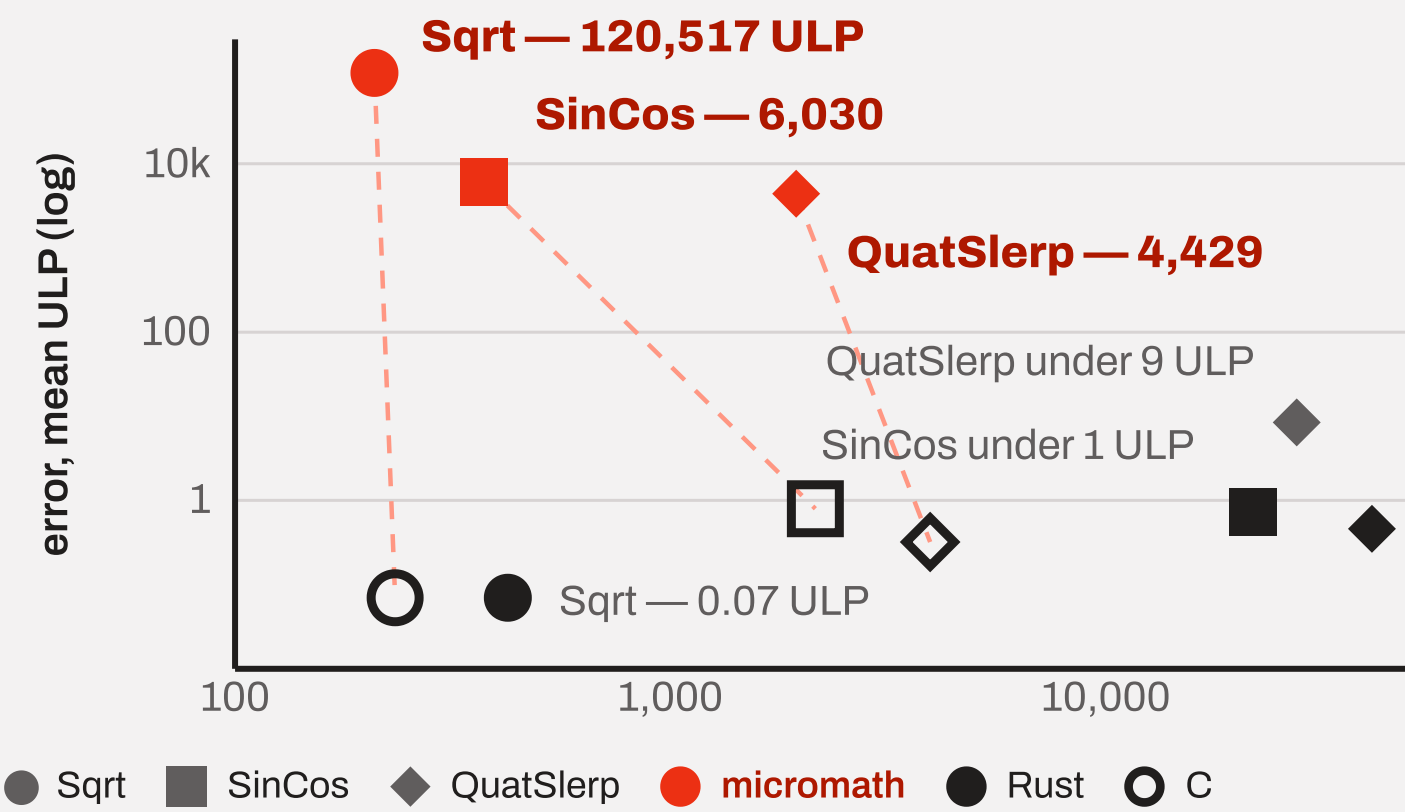
Each group is scaled to its own slowest bar. **newlib** is the ARM GCC toolchain's C math library.

WHY: ONE STEP ON THE FPU, OR MANY IN SOFTWARE



Square root splits the same way: newlib emits the single **vsqrt.f32** instruction; libm has no 32-bit ARM backend and falls back to a table plus integer iterations.

WHAT THE FAST ROUTE COSTS



Median execution time (ns) against mean ULP error, both log. Up and to the left is faster and less accurate; dashed lines join a fast-math routine to the one it replaces.

AND IT FLIPS THE OTHER WAY ON THREE OF THE FIVE

OPERATION	LIBM	NEWLIB
Exp	598	1,106
Ln	566	1,007
Atan2	1,091	1,456

Real code inherits whichever way its function went. **QuatSlerp** calls sine and cosine three times, so cmath3d runs **10.35×** faster than glam.

Across 18 primitives, 6 libraries, 4 chips and 4 build profiles, the language was the weakest predictor in the grid.

Matched on ABI class and profile, C and Rust land within 1.5× of each other — in both directions.

05 IF YOU ARE CHOOSING A MATH LIBRARY TODAY

- 01 A C-versus-Rust number is tied to the build profile that produced it.
- 02 A slow sqrt or sin/cos in Rust points at the linked math backend, not the language.
- 03 Fast-math crates trade orders of magnitude of accuracy for a few times the speed.

RUNNING NOW

Five boards run continuously in CI, including the nRF52840 added since the paper. The RP2350's RISC-V core is nearly ready.

NEXT

Closed-loop workloads: an EKF update and a geometric controller step — where ABI class, provider and accuracy tradeoff compound at once.

VERSIONS

glam 0.30.10 · nalgebra 0.35.0 · micromath 2.1.0 · libm 0.2.16 · CMSIS-DSP 1.17.1 · cmath3d 54f31e2. rustc 1.98.0, arm-none-eabi-gcc 14.2.1, clang 19.1.7.